

Numerical Schemes for SDE Simulation: Euler–Maruyama and Milstein

hasklib mathematical companion

Hubert

August 3, 2026

Abstract

This note derives the Euler–Maruyama scheme for simulating an Itô SDE, connects it directly to `EulerMaruyama::step/simulate_terminal`, and states its strong-order convergence result. It builds on the SDE framework and Brownian motion properties established in `ito_foundations.tex`. The Milstein scheme is then derived as a refinement: it captures a term Euler–Maruyama drops, using Itô’s lemma a second time, this time applied to the diffusion coefficient itself rather than to the process.

1 Setup: the problem discretisation solves

Recall from `ito_foundations.tex` the general Itô SDE

$$dX_t = a(t, X_t) dt + b(t, X_t) dW_t.$$

Integrating exactly over one interval $[t, t + \Delta t]$ gives

$$X_{t+\Delta t} - X_t = \int_t^{t+\Delta t} a(s, X_s) ds + \int_t^{t+\Delta t} b(s, X_s) dW_s. \quad (1)$$

This is exact, but neither integral can be evaluated in closed form in general, since a and b depend on X_s , which itself moves randomly across the interval. Simulation requires an approximation of (1).

2 The Euler–Maruyama scheme

Definition 1 (Euler–Maruyama update). The Euler–Maruyama approximation freezes a and b at their values at the start of the interval and approximates (1) by

$$\int_t^{t+\Delta t} a(s, X_s) ds \approx a(t, X_t) \Delta t, \quad \int_t^{t+\Delta t} b(s, X_s) dW_s \approx b(t, X_t)(W_{t+\Delta t} - W_t). \quad (2)$$

Remark 1 (Why freezing at t : the defining choice of an explicit scheme). By the mean value theorem for integrals, there exists some $\xi \in [t, t + \Delta t]$ (depending on the whole path) with

$$\int_t^{t+\Delta t} a(s, X_s) ds = a(\xi, X_\xi) \Delta t \quad \text{exactly.}$$

This is an exact statement but a useless one, since ξ and X_ξ are unknown until the interval has already been simulated. Euler–Maruyama’s approximation is to replace the unknown midpoint value $a(\xi, X_\xi)$ with the known left-endpoint value $a(t, X_t)$ — valid to leading order because a is

continuous and the interval is short, so $a(\xi, X_\xi)$ differs from $a(t, X_t)$ by an amount that shrinks as $\Delta t \rightarrow 0$ (quantified in Section 3 below). The diffusion approximation in (2) is the same idea applied to the stochastic integral: if $b(s, X_s)$ were genuinely constant over $[t, t + \Delta t]$, the stochastic integral would equal $b(t, X_t)(W_{t+\Delta t} - W_t)$ exactly, so freezing it at the left endpoint is again a leading-order replacement of an unknown quantity by a known one.

The choice of which endpoint to freeze at is what separates explicit from implicit schemes. Freezing at t means every quantity on the right of (2) is already known once we reach time t , so $X_{t+\Delta t}$ is computed by a single formula with no equation to solve. Freezing instead at the right endpoint $t+\Delta t$ (backward Euler) or at the midpoint would put the unknown $X_{t+\Delta t}$ (or an average involving it) on both sides of the update, requiring a root-find at every step. Explicitness is a computational convenience, not a mathematical requirement of the discretisation idea itself — it is a choice about which known quantity approximates $a(\xi, X_\xi)$, and `hasklib`'s Euler–Maruyama always chooses the one requiring no further work.

Using the Gaussian-increment property of Brownian motion (`ito_foundations.tex`, Definition 1), write $W_{t+\Delta t} - W_t = \sqrt{\Delta t} Z$ with $Z \sim \mathcal{N}(0, 1)$. Substituting into (2) and (1) gives the update rule:

$$X_{t+\Delta t} \approx X_t + a(t, X_t) \Delta t + b(t, X_t) \sqrt{\Delta t} Z. \quad (3)$$

Remark 2 (Correspondence with the code). Equation (3) is exactly `EulerMaruyama::step`:

```
double EulerMaruyama::step(const StochasticProcess& process, double t,
                           double x, double dt, double z) const
{
    double a = process.drift(t, x);
    double b = process.diffusion(t, x);
    return x + a * dt + b * std::sqrt(dt) * z;
}
```

Chaining (3) across N equal sub-intervals of $[0, T]$, with a fresh independent Z_i drawn at each step (the independent-increments property of Brownian motion), is exactly `EulerMaruyama::simulate_termina`

3 Why the approximation improves as $\Delta t \rightarrow 0$

The error introduced by Definition 1 comes entirely from treating $a(s, X_s)$ and $b(s, X_s)$ as constant over $[t, t + \Delta t]$, when in fact X_s moves within the interval. By the Brownian scaling argument in `ito_foundations.tex` (the nowhere-differentiability discussion), that movement is of order $\sqrt{\Delta t}$, so the error made by freezing the coefficients should itself shrink as $\Delta t \rightarrow 0$.

Proposition 1 (One-step (local) mean-square error). *Assume a, b are Lipschitz and continuously differentiable in x . Writing ε for the difference between the true increment (1) and the Euler–Maruyama approximation (2) over one step of length Δt , started from the same X_t ,*

$$\mathbb{E}[\varepsilon^2] = O((\Delta t)^2), \quad \text{i.e. the root-mean-square local error is } O(\Delta t).$$

Heuristic derivation. Split ε into its drift and diffusion contributions, $\varepsilon = \varepsilon_a + \varepsilon_b$, where

$$\varepsilon_a = \int_t^{t+\Delta t} (a(s, X_s) - a(t, X_t)) ds, \quad \varepsilon_b = \int_t^{t+\Delta t} (b(s, X_s) - b(t, X_t)) dW_s.$$

For $s \in [t, t + \Delta t]$, the true solution satisfies $X_s - X_t = a(t, X_t)(s - t) + b(t, X_t)(W_s - W_t) + \dots$; the first term is $O(\Delta t)$ and the second is $O(\sqrt{\Delta t})$ in an L^2 sense (its variance is $b(t, X_t)^2(s - t)$), so the Brownian term dominates and $X_s - X_t = O(\sqrt{\Delta t})$. By the Lipschitz/differentiability assumption, both coefficient differences inherit this order:

$$a(s, X_s) - a(t, X_t) = O(\sqrt{\Delta t}), \quad b(s, X_s) - b(t, X_t) = O(\sqrt{\Delta t}).$$

Drift term. Integrating a quantity of order $\sqrt{s-t}$ over $s \in [t, t+\Delta t]$ gives $\int_0^{\Delta t} \sqrt{u} du = \frac{2}{3}(\Delta t)^{3/2}$, so $\varepsilon_a = O((\Delta t)^{3/2})$ pathwise — already smaller than the diffusion contribution below, so it does not set the overall order.

Diffusion term. Here the integral is stochastic, so its size is governed by the (adapted) Itô isometry — the same fact used for the Moments of a Wiener Integral lemma in `ou.tex`, but for an integrand that is random and adapted rather than deterministic; see Shreve (2004) or Øksendal (2003) for this more general form:

$$\mathbb{E}[\varepsilon_b^2] = \mathbb{E} \left[\int_t^{t+\Delta t} (b(s, X_s) - b(t, X_t))^2 ds \right] \approx \int_t^{t+\Delta t} O(s-t) ds = O((\Delta t)^2),$$

using $\mathbb{E}[(b(s, X_s) - b(t, X_t))^2] = O(s-t)$ from the L^2 bound above. So ε_b has mean-square $O((\Delta t)^2)$, i.e. root-mean-square $O(\Delta t)$, which dominates the drift contribution and gives the proposition. $\square$

Remark 3. This is the mechanism, not yet the global result: a per-step error of root-mean-square order Δt does not immediately give a global error of the same order, because the errors from different steps compound through the process's dependence on X_t rather than simply adding. That compounding is the content of Theorem 1 below.

4 Strong convergence

Definition 2 (Strong convergence order). A scheme has *strong order* γ if there exists $C > 0$, independent of the step size Δt , such that

$$\mathbb{E}[|\hat{X}_T - X_T|] \leq C(\Delta t)^\gamma,$$

where $\hat{X}_T$ is the scheme's approximation and X_T is the true solution, evaluated pathwise on the same Brownian driving noise.

Theorem 1 (Strong order of Euler–Maruyama). *Under standard regularity conditions on a and b (Lipschitz continuity and linear growth, as in `ito_foundations.tex`, Assumption 1), the Euler–Maruyama scheme has strong order $\gamma = \frac{1}{2}$.*

Proof strategy. Proposition 1 gives a per-step root-mean-square error of order Δt . Simulating to time T takes $N = T/\Delta t$ such steps, and the natural (but wrong) guess is that independent per-step errors of size Δt accumulate to a global error of size $N \cdot \Delta t = T$ — a constant, not something that shrinks. That guess is wrong because it ignores that each step's error is fed back through a and b at the start of the next step: an error made at step i perturbs X_{t_i} , which perturbs every subsequent coefficient evaluation. Controlling this requires a discrete Grönwall-type argument: the Lipschitz bound on a and b shows that an error present at step i can grow by at most a bounded multiplicative factor per subsequent step, so the N accumulated local errors combine as a sum dominated by the largest few terms rather than by their count. Carrying this through converts the local root-mean-square bound $O(\Delta t)$ into a global strong error of $O(\sqrt{N} \cdot \Delta t) = O(\sqrt{T/\Delta t} \cdot \Delta t) = O(\sqrt{\Delta t})$, i.e. strong order $\frac{1}{2}$. The technical machinery (the precise Grönwall estimate and the regularity conditions under which it holds) is in Kloeden & Platen (1992), Ch. 9–10; we cite the result rather than reproduce the estimate. $\square$

Remark 4 (Correspondence with the code). This is precisely what `test_stochastic.cpp`'s convergence block checks empirically: running `simulate_terminal` on GBM at increasing step counts $N \in \{10, 50, 200, 1000\}$ and observing the empirical terminal moments approach the exact moments from `GBM::sample_terminal` as $\Delta t = T/N \rightarrow 0$. Theorem 1 is the theoretical statement of which the test's shrinking printed errors are a numerical demonstration.

Remark 5 (Contrast with the exact sampler). For GBM specifically, `GBM::sample_terminal` has zero discretisation error, because the SDE admits a closed-form solution via Itô's lemma (see `gbm.tex`). Euler–Maruyama is an approximation of the same SDE and is only needed in general because most processes — and, in particular, any path-dependent payoff even for GBM — do not admit such a closed form.

5 The Milstein scheme

Euler–Maruyama's error, from Proposition 1, is dominated entirely by the diffusion contribution ε_b — the drift contribution was already a full order smaller. Milstein's idea is to attack exactly that term: instead of freezing $b(s, X_s)$ at its start-of-interval value the way (2) does, expand it one order further using Itô's lemma applied to b itself, and keep the next term the expansion produces.

Lemma 1 (A stochastic integral Itô's lemma evaluates exactly). *For a step from t to $t + \Delta t$, writing $\Delta W = W_{t+\Delta t} - W_t$,*

$$\int_t^{t+\Delta t} (W_s - W_t) dW_s = \frac{1}{2} [(\Delta W)^2 - \Delta t].$$

Proof. Let $\widetilde{W}_s = W_s - W_t$ for $s \geq t$; this is itself a standard Brownian motion started at 0 at time t . Apply Itô's lemma (Theorem 1 of `ito_foundations.tex`) to $f(x) = \frac{1}{2}x^2$ along $\widetilde{W}_s$ (drift 0, diffusion 1): $f_x = x$, $f_{xx} = 1$, so

$$d(\frac{1}{2}\widetilde{W}_s^2) = \left(0 + \frac{1}{2} \cdot 1\right)ds + \widetilde{W}_s dW_s = \frac{1}{2}ds + \widetilde{W}_s dW_s.$$

Integrating from t to $t + \Delta t$, using $\widetilde{W}_t = 0$ and $\widetilde{W}_{t+\Delta t} = \Delta W$,

$$\frac{1}{2}(\Delta W)^2 - 0 = \frac{1}{2}\Delta t + \int_t^{t+\Delta t} (W_s - W_t) dW_s,$$

which rearranges to the claimed identity. □

Definition 3 (Milstein update). Expand $b(s, X_s)$ around (t, X_t) to first order in x (dropping the t -derivative, which contributes only at higher order once multiplied against dW_s):

$$b(s, X_s) \approx b(t, X_t) + b_x(t, X_t) (X_s - X_t).$$

Keeping only the leading (Brownian) part of $X_s - X_t \approx b(t, X_t)(W_s - W_t)$ and substituting into the diffusion integral of (1),

$$\int_t^{t+\Delta t} b(s, X_s) dW_s \approx b(t, X_t) \int_t^{t+\Delta t} dW_s + b(t, X_t) b_x(t, X_t) \int_t^{t+\Delta t} (W_s - W_t) dW_s.$$

The first integral is $\Delta W = \sqrt{\Delta t} Z$, as in Euler–Maruyama; the second is exactly Lemma 1. Substituting both and writing $\Delta W = \sqrt{\Delta t} Z$ throughout gives the *Milstein update*:

$$X_{t+\Delta t} \approx X_t + a(t, X_t) \Delta t + b(t, X_t) \sqrt{\Delta t} Z + \frac{1}{2} b(t, X_t) b_x(t, X_t) \Delta t (Z^2 - 1). \quad (4)$$

Remark 6 (Correspondence with the code). Equation (4) is exactly `Milstein::step`: the same `a`, `b` evaluations as `EulerMaruyama::step`, plus one further call — the diffusion's x -derivative, `process.diffusion_derivative(t,x)` — and the extra correction term `0.5 * b * b_x * dt * (z*z - 1)` added to the Euler–Maruyama result. `Milstein::simulate_terminal` chains this update the same way `EulerMaruyama::simulate_terminal` does.

Theorem 2 (Strong order of Milstein). *Under the same regularity conditions as Theorem 1, together with b being twice continuously differentiable in x , the Milstein scheme has strong order $\gamma = 1$.*

Proof strategy. Lemma 1 shows the correction term recovers exactly the contribution Euler–Maruyama’s freezing of b discarded — the term responsible for ε_b ’s $O(\Delta t)$ root-mean-square local error in Proposition 1. Capturing it removes the leading source of error, leaving a local error one order smaller; accumulating $N = T/\Delta t$ such steps by the same compounding argument as Theorem 1 yields a global strong error of $O(\Delta t)$ rather than $O(\sqrt{\Delta t})$. The technical estimates are in Kloeden & Platen (1992), Ch. 10; we cite the result rather than reproduce it. $\square$

Remark 7 (A free regression check: constant diffusion). Whenever $b(t, x)$ does not depend on x — ABM’s constant σ , or OU’s constant σ — the derivative $b_x(t, x) = 0$ identically, so the entire correction term in (4) vanishes exactly, not just approximately. On those two processes, `Milstein::step` should therefore return bit-for-bit the same value as `EulerMaruyama::step` given the same $t, x, \Delta t, z$ — a direct, zero-tolerance regression test requiring no statistical comparison, only that `diffusion_derivative` correctly returns 0 for those two classes.

Remark 8 (Where the correction is not free: GBM and CIR). For GBM, $b(t, x) = \sigma x$ gives $b_x = \sigma$, a genuine nonzero correction $\frac{1}{2}\sigma^2 X_t \Delta t (Z^2 - 1)$ — though GBM already has an exact sampler (`gbm.tex`), so Milstein only matters there for path-dependent payoffs, not terminal sampling. For CIR, $b(t, x) = \sigma\sqrt{x}$ gives $b_x = \sigma/(2\sqrt{x})$, and the correction is both nonzero and, like Euler–Maruyama applied to CIR (`cir.tex`), vulnerable to the same well-definedness failure if a step pushes X_t negative before the next $\sqrt{x}$ evaluation. Whether Milstein on CIR needs its own truncation fix analogous to full truncation is a real question this note does not resolve.

References

- Kloeden, P. E. & Platen, E. (1992), *Numerical Solution of Stochastic Differential Equations*, Vol. 23 of *Applications of Mathematics*, Springer, Berlin.
- Øksendal, B. (2003), *Stochastic Differential Equations: An Introduction with Applications*, 6th edn, Springer, Berlin.
- Shreve, S. E. (2004), *Stochastic Calculus for Finance II: Continuous-Time Models*, Springer Finance, Springer, New York.